

""

Experiment 1 – SEAG vs. Interface-Layer
Baseline under simulated failure conditions
(Section 7.1 of the SAW paper)

This module implements TWO REAL, RUNNABLE
systems:

1. InterfaceLayerSystem – approval is
enforced only by application/UI convention.

The execution function can be called
directly; nothing in the system prevents
a bypass, and a "confirmed" click does
not have to be durably persisted before
execution proceeds. This models the
"performative" approval pattern described
in Section 8.1 of the paper.

2. SEAGSystem – approval is enforced via a
SQLite database with a real FOREIGN KEY
constraint (approved_by REFERENCES
users(id)) and an explicit state machine
check (running -> awaiting_approval ->
executed) performed inside a DB
transaction before the execution
function is ever reachable.

All measurements in the accompanying
experiment_runner.py come from actually
running these systems under failure

```

injection – no numbers are invented.
"""

import sqlite3
import threading
import random
import time
import uuid

def fresh_db(path=":memory:"):
    conn = sqlite3.connect(path,
check_same_thread=False)
    conn.execute("PRAGMA foreign_keys =
ON;")
    conn.execute("""
        CREATE TABLE users (
            id INTEGER PRIMARY KEY,
            name TEXT NOT NULL
        )
    """)
    conn.execute("""
        CREATE TABLE workflow_runs (
            id TEXT PRIMARY KEY,
            status TEXT NOT NULL DEFAULT
'running',
            approved_by INTEGER,
            executed_flag INTEGER NOT NULL
DEFAULT 0,
            FOREIGN KEY (approved_by)
REFERENCES users(id)
        )
    """)
    conn.execute("INSERT INTO users (id,

```

```

name) VALUES (1, 'authenticated_user'))
    conn.commit()
    return conn

class ExecutionLog:
    """Ground-truth audit log shared by
    both systems, used only for measurement."""
    def __init__(self):
        self.lock = threading.Lock()
        self.executions = [] # list of
        dicts: {run_id, had_valid_approval, ts}

    def record(self, run_id,
        had_valid_approval):
        with self.lock:
            self.executions.append({
                "run_id": run_id,
                "had_valid_approval":
        had_valid_approval,
                "ts": time.time(),
            })

# -----
# -----
# SYSTEM 1: Interface-Layer Baseline
# (performative approval)
# -----
# -----

class InterfaceLayerSystem:
    """
    Approval is a frontend confirmation
    dialog. The 'approved' state lives in an

```

in-memory client-side flag that is NOT required to be durably persisted before the execute() action fires. This mirrors real interface-layer designs where the click handler calls execute_action() optimistically, independent of whether the backend confirmation call actually succeeded.

```
"""
def __init__(self, log: ExecutionLog):
    self.log = log
    self.runs = {} # run_id ->
{'status': str}
```

```
def create_run(self):
    run_id = str(uuid.uuid4())
    self.runs[run_id] = {"status":
"running"}
    return run_id
```

```
def present_confirmation_dialog(self,
run_id):
    self.runs[run_id]["status"] =
"awaiting_approval"
```

```
def user_clicks_approve(self, run_id,
backend_call_succeeds=True):
```

```
"""
```

User clicks 'Approve'. In a real system this triggers:

- (a) a backend call to persist the approval, AND
- (b) a frontend callback that

```

fires execute_action()
    These are two separate network
events. If (a) fails but the frontend
    still calls (b) [a very common
real-world bug: optimistic UI], execution
    proceeds without a persisted
approval record.
    """
    approved_persisted =
backend_call_succeeds
    if approved_persisted:
        self.runs[run_id]["status"] =
"executed"
        # Frontend fires execution
regardless of persistence success
        # (this is the performative-
approval bug being tested)
        self.execute_action(run_id,
had_valid_approval=approved_persisted)

    def bypass_execute_directly(self,
run_id):
        """Simulates an automated client
calling the execution endpoint directly,
        skipping the confirmation dialog
entirely."""
        self.execute_action(run_id,
had_valid_approval=False)

    def execute_action(self, run_id,
had_valid_approval):
        """No precondition check exists in
this design – execute_action is a
        plain function reachable from

```

```

anywhere in the code."""
        self.log.record(run_id,
had_valid_approval)

# -----
# SYSTEM 2: SEAG (Schema-Enforced Approval
Gating)
# -----

class SEAGSystem:
    """
        Approval is enforced by a SQLite-backed
state machine with a real FK
constraint. execute_action() is only
reachable from inside approve_workflow(),
AFTER the DB transaction that sets
status='executed' and approved_by=<valid
user>
has committed successfully. There is no
code path to execute_action() that
does not go through this function.
    """

    def __init__(self, log: ExecutionLog,
conn: sqlite3.Connection):
        self.log = log
        self.conn = conn
        self.db_lock = threading.Lock() #
models a single DB connection's
serialization

    def create_run(self):
        run_id = str(uuid.uuid4())

```

```

        with self.db_lock:
            self.conn.execute(
                "INSERT INTO workflow_runs
(id, status) VALUES (?, 'running')",
(run_id,)
            )
            self.conn.commit()
        return run_id

    def present_confirmation_dialog(self,
run_id):
        with self.db_lock:
            self.conn.execute(
                "UPDATE workflow_runs SET
status='awaiting_approval' WHERE id=?",
(run_id,)
            )
            self.conn.commit()

    def approve_workflow(self, run_id,
user_id=1, network_fails=False):
        """
        Mirrors the /workflow/approve
endpoint in the paper (Section 4.2).
        Precondition check + FK-constrained
state transition inside one
transaction; execute_action() is
only called after a successful commit.
        """
        if network_fails:
            # The approval request never
reaches the server (or times out before
            # response). No DB write
happens. Nothing downstream can execute.

```

```

        return False

        with self.db_lock:
            cur = self.conn.execute(
                "SELECT status FROM
workflow_runs WHERE id=?", (run_id,)
            )
            row = cur.fetchone()
            if row is None or row[0] !=
"awaiting_approval":
                # Precondition check fails
-> reject
                return False
            try:
                self.conn.execute(
                    "UPDATE workflow_runs
SET status='executed', approved_by=? "
                    "WHERE id=? AND
status='awaiting_approval'",
                    (user_id, run_id),
                )
                self.conn.commit()
            except sqlite3.IntegrityError:
                # FK constraint violation
(invalid user) -> reject
                self.conn.rollback()
                return False

            # Verify the write actually
took effect (guards against races where
            # another thread already
flipped the status between SELECT and
UPDATE)
            cur2 = self.conn.execute(

```



```

        "SELECT status, approved_by
FROM workflow_runs WHERE id=?", (run_id,)
    )
    status, approved_by =
cur2.fetchone()
    if status == "executed" and
approved_by == user_id:
        self.execute_action(run_id,
had_valid_approval=True)
        return True
    return False

def bypass_execute_directly(self,
run_id):
    """
    Simulates an attacker trying to
call the execution function directly.
    In the real SAW codebase this is
structurally impossible because
    execute_workflow_actions() is a
private function only invoked from
    inside approve_workflow(). Here we
simulate the attacker instead
    checking the DB state themselves
before deciding to (illegitimately)
    invoke execution logic – i.e., we
give the bypass attempt the *same*
    DB-gated entry point SEAG provides,
to test whether the gate holds.
    """
    with self.db_lock:
        cur = self.conn.execute(
            "SELECT status, approved_by
FROM workflow_runs WHERE id=?", (run_id,)

```

```
        )
        row = cur.fetchone()
        if row and row[0] == "executed" and
row[1] is not None:
            self.execute_action(run_id,
had_valid_approval=True)
        else:
            # No valid persisted approval -
> SEAG blocks; nothing executes.
            pass

    def execute_action(self, run_id,
had_valid_approval):
        self.log.record(run_id,
had_valid_approval)
```